

Swagger Petstore

- **Swagger Version:** 2.0
- **API Version:** 1.0.7

This is a sample server Petstore server. You can find out more about Swagger at <http://swagger.io> or on irc.freenode.net, [#swagger](https://www.slack.com/join/shared_invite/zt-1000000000-0000000000) . For this sample, you can use the api key `special-key` to test the authorization filters.

Servers

- `https://petstore.swagger.io/v2`
- `http://petstore.swagger.io/v2`

pet

Everything about your Pets

[Find out more](#)

POST `/pet/{petId}/uploadImage`

uploads an image

Parameters

Name	Description
petId <i>required</i> integer(\$int64) (path)	ID of pet to update
additionalMetadata string (formData)	Additional data to pass to server
file file (formData)	file to upload

Responses

Response content type: application/json

Code	Description
200	successful operation

Example Value

Status Code: 200

```
{
  "code": 0,
  "type": "string",
  "message": "string"
}
```

Model

Property	Type	Required	Description
code	integer(\$int32)	no	none
type	string	no	none

Property	Type	Required	Description
message	string	no	none

POST /pet

Add a new pet to the store

Parameters

Name	Description
body <i>required</i> object (body)	Pet object that needs to be added to the store

Request Body

Pet object that needs to be added to the store

Content-Type: application/json

Property	Type	Required	Description
id	integer(\$int64)	no	none
category	object	no	none
name	string	yes	none
photoUrls	array[string]	yes	none
tags	array[object]	no	none
status	string	no	pet status in the store

Example Value

```
{
  "id": 0,
  "category": {
    "id": 0,
    "name": "string"
  },
  "name": "doggie",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Content-Type: application/xml

Property	Type	Required	Description
id	integer(\$int64)	no	none
category	object	no	none
name	string	yes	none
photoUrls	array[string]	yes	none
tags	array[object]	no	none
status	string	no	pet status in the store

Example Value

```
<!-- XML example not generated -->
```

Responses

Response content type: `application/json`

Code	Description
405	Invalid input

PUT `/pet`

Update an existing pet

Parameters

Name	Description
<code>body</code> <i>required</i> object (body)	Pet object that needs to be added to the store

Request Body

Pet object that needs to be added to the store

Content-Type: `application/json`

Property	Type	Required	Description
<code>id</code>	<code>integer(\$int64)</code>	no	none
<code>category</code>	object	no	none

Property	Type	Required	Description
name	string	yes	none
photoUrls	array[string]	yes	none
tags	array[object]	no	none
status	string	no	pet status in the store

Example Value

```
{
  "id": 0,
  "category": {
    "id": 0,
    "name": "string"
  },
  "name": "doggie",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Content-Type: application/xml

Property	Type	Required	Description
id	integer(\$int64)	no	none
category	object	no	none
name	string	yes	none
photoUrls	array[string]	yes	none
tags	array[object]	no	none

Property	Type	Required	Description
status	string	no	pet status in the store

Example Value

```
<!-- XML example not generated -->
```

Responses

Response content type: application/json

Code	Description
400	Invalid ID supplied
404	Pet not found
405	Validation exception

GET /pet/findByStatus

Finds Pets by status

Multiple status values can be provided with comma separated strings

Parameters

Name	Description
status <i>required</i> array[string]	Status values that need to be considered for filter

Name	Description
(query)	

Responses

Response content type: application/json

Code	Description
200	successful operation
400	Invalid status value

Example Value

Status Code: 200

```
[
  {
    "id": 0,
    "category": {
      "id": 0,
      "name": "string"
    },
    "name": "doggie",
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 0,
        "name": "string"
      }
    ],
    "status": "available"
  }
]
```


Model

Array of:

Property	Type	Required	Description
id	integer(\$int64)	no	none
category	object	no	none
name	string	yes	none
photoUrls	array[string]	yes	none
tags	array[object]	no	none
status	string	no	pet status in the store

GET /pet/findByTags

Finds Pets by tags

Warning

This operation is deprecated.

Multiple tags can be provided with comma separated strings. Use tag1, tag2, tag3 for testing.

Parameters

Name	Description
tags <i>required</i> array[string] (query)	Tags to filter by

Responses

Response content type: application/json

Code	Description
200	successful operation
400	Invalid tag value

Example Value

Status Code: 200

```
[
  {
    "id": 0,
    "category": {
      "id": 0,
      "name": "string"
    },
    "name": "doggie",
    "photoUrls": [
      "string"
    ],
    "tags": [
      {
        "id": 0,
        "name": "string"
      }
    ],
    "status": "available"
  }
]
```

Model

Array of:

Property	Type	Required	Description
id	integer(\$int64)	no	none
category	object	no	none
name	string	yes	none
photoUrls	array[string]	yes	none
tags	array[object]	no	none
status	string	no	pet status in the store

GET /pet/{petId}

Find pet by ID

Returns a single pet

Parameters

Name	Description
petId <i>required</i> integer(\$int64) (path)	ID of pet to return

Responses

Response content type: application/json

Code	Description
200	successful operation

Code	Description
400	Invalid ID supplied
404	Pet not found

Example Value

Status Code: 200

```
{
  "id": 0,
  "category": {
    "id": 0,
    "name": "string"
  },
  "name": "doggie",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Model

Property	Type	Required	Description
id	integer(\$int64)	no	none
category	object	no	none
name	string	yes	none
photoUrls	array[string]	yes	none
tags	array[object]	no	none

Property	Type	Required	Description
status	string	no	pet status in the store

POST /pet/{petId}

Updates a pet in the store with form data

Parameters

Name	Description
petId <i>required</i> integer(\$int64) (path)	ID of pet that needs to be updated
name string (formData)	Updated name of the pet
status string (formData)	Updated status of the pet

Responses

Response content type: application/json

Code	Description
405	Invalid input

DELETE /pet/{petId}

Deletes a pet

Parameters

Name	Description
api_key string (header)	none
petId <i>required</i> integer(\$int64) (path)	Pet id to delete

Responses

Response content type: application/json

Code	Description
400	Invalid ID supplied
404	Pet not found

store

Access to Petstore orders

GET /store/inventory

Returns pet inventories by status

Returns a map of status codes to quantities

Responses

Response content type: `application/json`

Code	Description
200	successful operation

Example Value

Status Code: `200`

```
{}
```

Model

`object`

POST `/store/order`

Place an order for a pet

Parameters

Name	Description
body <i>required</i> object (body)	order placed for purchasing the pet

Request Body

order placed for purchasing the pet

Content-Type: application/json

Property	Type	Required	Description
id	integer(\$int64)	no	none
petId	integer(\$int64)	no	none
quantity	integer(\$int32)	no	none
shipDate	string(\$date-time)	no	none
status	string	no	Order Status
complete	boolean	no	none

Example Value

```
{
  "id": 0,
  "petId": 0,
  "quantity": 0,
  "shipDate": "2026-01-01T00:00:00Z",
  "status": "placed",
  "complete": true
}
```

Responses

Response content type: application/json

Code	Description
200	successful operation
400	Invalid Order

Example Value

Status Code: 200

```
{
  "id": 0,
  "petId": 0,
  "quantity": 0,
  "shipDate": "2026-01-01T00:00:00Z",
  "status": "placed",
  "complete": true
}
```

Model

Property	Type	Required	Description
id	integer(\$int64)	no	none
petId	integer(\$int64)	no	none
quantity	integer(\$int32)	no	none
shipDate	string(\$date-time)	no	none
status	string	no	Order Status
complete	boolean	no	none

GET /store/order/{orderId}

Find purchase order by ID

For valid response try integer IDs with value ≥ 1 and ≤ 10 . Other values will generated exceptions

Parameters

Name	Description
<code>orderId</code> <i>required</i> <code>integer(\$int64)</code> (path)	ID of pet that needs to be fetched

Responses

Response content type: `application/json`

Code	Description
200	successful operation
400	Invalid ID supplied
404	Order not found

Example Value

Status Code: 200

```
{
  "id": 0,
  "petId": 0,
  "quantity": 0,
  "shipDate": "2026-01-01T00:00:00Z",
  "status": "placed",
```

```
"complete": true
}
```

Model

Property	Type	Required	Description
id	integer(\$int64)	no	none
petId	integer(\$int64)	no	none
quantity	integer(\$int32)	no	none
shipDate	string(\$date-time)	no	none
status	string	no	Order Status
complete	boolean	no	none

DELETE /store/order/{orderId}

Delete purchase order by ID

For valid response try integer IDs with positive integer value. Negative or non-integer values will generate API errors

Parameters

Name	Description
orderId <i>required</i> integer(\$int64) (path)	ID of the order that needs to be deleted

Responses

Response content type: `application/json`

Code	Description
400	Invalid ID supplied
404	Order not found

user

Operations about user

[Find out more about our store](#)

POST `/user/createWithList`

Creates list of users with given input array

Parameters

Name	Description
<code>body</code> <i>required</i> <code>array[object]</code> (body)	List of user object

Request Body

List of user object

Content-Type: application/json

Array of:

Property	Type	Required	Description
id	integer(\$int64)	no	none
username	string	no	none
firstName	string	no	none
lastName	string	no	none
email	string	no	none
password	string	no	none
phone	string	no	none
userStatus	integer(\$int32)	no	User Status

Example Value

```
[
  {
    "id": 0,
    "username": "string",
    "firstName": "string",
    "lastName": "string",
    "email": "string",
    "password": "string",
    "phone": "string",
    "userStatus": 0
  }
]
```

Responses

Response content type: application/json

Code	Description
default	successful operation

GET /user/{username}

Get user by user name

Parameters

Name	Description
username <i>required</i> string (path)	The name that needs to be fetched. Use user1 for testing.

Responses

Response content type: application/json

Code	Description
200	successful operation
400	Invalid username supplied
404	User not found

Example Value

Status Code: 200

```
{
  "id": 0,
  "username": "string",
  "firstName": "string",
  "lastName": "string",
  "email": "string",
  "password": "string",
  "phone": "string",
  "userStatus": 0
}
```

Model

Property	Type	Required	Description
id	integer(\$int64)	no	none
username	string	no	none
firstName	string	no	none
lastName	string	no	none
email	string	no	none
password	string	no	none
phone	string	no	none
userStatus	integer(\$int32)	no	User Status

PUT /user/{username}

Updated user

This can only be done by the logged in user.

Parameters

Name	Description
username <i>required</i> string (path)	name that need to be updated
body <i>required</i> object (body)	Updated user object

Request Body

Updated user object

Content-Type: application/json

Property	Type	Required	Description
id	integer(\$int64)	no	none
username	string	no	none
firstName	string	no	none
lastName	string	no	none
email	string	no	none
password	string	no	none
phone	string	no	none
userStatus	integer(\$int32)	no	User Status

Example Value

```
{  
  "id": 0,  
  "username": "string",
```



```
"firstName": "string",
"lastName": "string",
"email": "string",
"password": "string",
"phone": "string",
"userStatus": 0
}
```

Responses

Response content type: application/json

Code	Description
400	Invalid user supplied
404	User not found

DELETE /user/{username}

Delete user

This can only be done by the logged in user.

Parameters

Name	Description
username <i>required</i> string (path)	The name that needs to be deleted

Responses

Response content type: `application/json`

Code	Description
400	Invalid username supplied
404	User not found

GET `/user/login`

Logs user into the system

Parameters

Name	Description
<code>username</code> <i>required</i> string (query)	The user name for login
<code>password</code> <i>required</i> string (query)	The password for login in clear text

Responses

Response content type: `application/json`

Code	Description
200	successful operation
400	Invalid username/password supplied

Example Value

Status Code: 200

```
"string"
```

Model

```
string
```

GET /user/logout

Logs out current logged in user session

Responses

Response content type: application/json

Code	Description
default	successful operation

POST /user/createWithArray

Creates list of users with given input array

Parameters

Name	Description
<code>body</code> <i>required</i> <code>array[object]</code> (<code>body</code>)	List of user object

Request Body

List of user object

Content-Type: `application/json`

Array of:

Property	Type	Required	Description
<code>id</code>	<code>integer(\$int64)</code>	no	none
<code>username</code>	<code>string</code>	no	none
<code>firstName</code>	<code>string</code>	no	none
<code>lastName</code>	<code>string</code>	no	none
<code>email</code>	<code>string</code>	no	none
<code>password</code>	<code>string</code>	no	none
<code>phone</code>	<code>string</code>	no	none
<code>userStatus</code>	<code>integer(\$int32)</code>	no	User Status

Example Value

```
[
  {
    "id": 0,
    "username": "string",
    "firstName": "string",
    "lastName": "string",
    "email": "string",
    "password": "string",
    "phone": "string",
    "userStatus": 0
  }
]
```

Responses

Response content type: application/json

Code	Description
default	successful operation

POST /user

Create user

This can only be done by the logged in user.

Parameters

Name	Description
body <i>required</i> object (body)	Created user object

Request Body

Created user object

Content-Type: application/json

Property	Type	Required	Description
id	integer(\$int64)	no	none
username	string	no	none
firstName	string	no	none
lastName	string	no	none
email	string	no	none
password	string	no	none
phone	string	no	none
userStatus	integer(\$int32)	no	User Status

Example Value

```
{
  "id": 0,
  "username": "string",
  "firstName": "string",
  "lastName": "string",
  "email": "string",
  "password": "string",
  "phone": "string",
  "userStatus": 0
}
```

Responses

Response content type: application/json

Code	Description
default	successful operation

Schemas

ApiResponse

- **Type:** object

Property	Type	Required	Description
code	integer(\$int32)	no	none
type	string	no	none
message	string	no	none

Example Value

```
{
  "code": 0,
  "type": "string",
  "message": "string"
}
```

Category

- **Type:** object

Property	Type	Required	Description
id	integer(\$int64)	no	none
name	string	no	none

Example Value

```
{
  "id": 0,
  "name": "string"
}
```

Pet

- **Type:** object

Property	Type	Required	Description
id	integer(\$int64)	no	none
category	object	no	none
name	string	yes	none
photoUrls	array[string]	yes	none
tags	array[object]	no	none
status	string	no	pet status in the store

Example Value

```
{
  "id": 0,
  "category": {
    "id": 0,
```



```
    "name": "string"
  },
  "name": "doggie",
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Tag

- **Type:** object

Property	Type	Required	Description
id	integer(\$int64)	no	none
name	string	no	none

Example Value

```
{
  "id": 0,
  "name": "string"
}
```

Order

- **Type:** object

Property	Type	Required	Description
id	integer(\$int64)	no	none
petId	integer(\$int64)	no	none
quantity	integer(\$int32)	no	none
shipDate	string(\$date-time)	no	none
status	string	no	Order Status
complete	boolean	no	none

Example Value

```
{
  "id": 0,
  "petId": 0,
  "quantity": 0,
  "shipDate": "2026-01-01T00:00:00Z",
  "status": "placed",
  "complete": true
}
```

User

- **Type:** object

Property	Type	Required	Description
id	integer(\$int64)	no	none
username	string	no	none
firstName	string	no	none

Property	Type	Required	Description
lastName	string	no	none
email	string	no	none
password	string	no	none
phone	string	no	none
userStatus	integer(\$int32)	no	User Status

Example Value

```
{
  "id": 0,
  "username": "string",
  "firstName": "string",
  "lastName": "string",
  "email": "string",
  "password": "string",
  "phone": "string",
  "userStatus": 0
}
```